

TASK 1: Examining data types

The first task is to identify mismatched data types and make the data type change appropriate. It is essential to compare the correct data types and original data types of the variables.

1.1. Comparison between Table 1 and original data type

No	Column name	Description
1	Participant_ID	Identity number of participants
2	Survey_date	Recorded date of the data
3	Region	Region code, e.g., 'N-A' stands for North America
4	Country	Country code, e.g., 'AU' stands for Australia
5	Ip_latitude	Latitude coordinate of the location
6	Ip_longitude	Longitude coordinate of the location
7	Gender	Male, Female or other
8	Age	Age quantile
9	Height	Height of the person in cm
10	Weight	Weight of the person in kg
11	Bmi	Body Mass Index
12	Blood_type	Type of the person's blood
13	Insurance	If the person has insurance or not?
14	Income	Type of income. For example, low, medium, high, or gov
15	Smoking	Information on how often the person smokes
16	alcohol	Level of alcohol
17	cocaine	Level of cocaine
18	Contacts_count	Number of people the person has contacted
19	Public_transport_count	Number of people contacted by the person during public transportation
20	Working	Status of the person's work
21	Worried	On a scale of 1 to 5 indicating how much worried is the person?
22	covid19_positive	1 and 0 stating the existence or not
23	covid19_symptoms	
24	covid19_contact	
25	asthma	
26	kidney_disease	
27	liver_disease	
28	compromised_immune	
29	heart_disease	
30	lung_disease	
31	diabetes	
32	hiv_positive	
33	other_chronic	
34	nursing_home	
35	health_worker	
36	risk_infection	Risk of the person to get infected
37	risk_infection_level	3 risk levels of the person to get infected: high, moderate, low
38	risk_mortality	Risk of the person to die


```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5789 entries, 0 to 5788
Data columns (total 38 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Participant_ID                        5789 non-null   int64
1   survey_date                          5789 non-null   object
2   region                              5774 non-null   object
3   country                             5786 non-null   object
4   ip_latitude                          5789 non-null   float64
5   ip_longitude                        5789 non-null   float64
6   gender                              5789 non-null   object
7   age                                 5789 non-null   object
8   height                              5789 non-null   int64
9   weight                              5789 non-null   int64
10  bmi                                 5789 non-null   float64
11  blood_type                          5689 non-null   object
12  insurance                           4497 non-null   object
13  income                             4636 non-null   object
14  smoking                            5753 non-null   object
15  alcohol                            5789 non-null   float64
16  cocaine                             1430 non-null   float64
17  contacts_count                      5747 non-null   float64
18  public_transport_count              4572 non-null   float64
19  working                             5761 non-null   object
20  worried                             4518 non-null   float64
21  covid19_positive                   5789 non-null   int64
22  covid19_symptoms                   5789 non-null   int64
23  covid19_contact                    5789 non-null   int64
24  asthma                             5789 non-null   int64
25  kidney_disease                     5789 non-null   int64
26  liver_disease                      5789 non-null   int64
27  compromised_immune                 5789 non-null   int64
28  heart_disease                      5789 non-null   int64
29  lung_disease                       5789 non-null   int64
30  diabetes                           5789 non-null   int64
31  hiv_positive                       5789 non-null   int64
32  other_chronic                      5789 non-null   int64
33  nursing_home                       5789 non-null   int64
34  health_worker                      5789 non-null   int64
35  risk_infection                     5789 non-null   float64
36  risk_infection_level               5789 non-null   object
37  risk_mortality                     5789 non-null   float64
dtypes: float64(10), int64(17), object(11)
memory usage: 1.7+ MB
None

```

Table 1.1. correct data types in assessment detail

Table 1.2. original data type

There are four incorrect data types of variables according to the result of comparison with Table 1.1 and Table 1.2.

Variable Name	Original Type	Expected Type	Reason
survey_date	object	datetime	To recognise and handle datetime data in data analysis.
height	int	float	Height may have a floating number for accuracy.
weight	int	float	Weight may have a floating number for accuracy.
insurance	object	binary	'yes', 'no' can be converted to binary values for convenient data analysis.

Insurance Name: insurance, dtype: object [nan 'yes' 'no' 'blank'] insurance yes 3544 no 622 blank 331 Name: count, dtype: int64	=====insurance===== insurance 1.0 3544 0.0 622 Name: count, dtype: int64 ----- [nan 1. 0.]	'insurance' values were changed to binary values using map() function. Therefore, other values except for 'yes' and 'no' were already changed to NaN. Therefore, it is unnecessary to change 'blank' to 'nan'
--	--	---

1.2. Corrected Data Types

Python code snippets & outputs	Correct data types
<pre>df['survey_date'] = pd.to_datetime(df['survey_date'], errors='coerce') df['height'] = df['height'].astype(float) df['weight'] = df['weight'].astype(float)</pre> How to change - 'survey_date': convert from object to datetime using 'pd.to_datetime()' - 'height', 'weight': change int to float because of the possibility of float number <pre>[3]: df['insurance'].value_counts()</pre> <pre>[3]: insurance yes 3544 no 622 blank 331 Name: count, dtype: int64</pre> <pre>[6]: insurance_map = {'yes': 1, 'no': 0} df['insurance'] = df['insurance'].map(insurance_map)</pre> <pre>[7]: df['insurance'].value_counts()</pre> <pre>[7]: insurance 1.0 3544 0.0 622 Name: count, dtype: int64</pre> How to change - 'insurance' valuable has 'yes','no', 'blank': convert 'yes': 1 and 'no': 0 using 'map()' to represent a binary value. - 'blank' will be addressed in Task3.	<pre><class 'pandas.core.frame.DataFrame'> RangeIndex: 5789 entries, 0 to 5788 Data columns (total 38 columns): # Column Non-Null Count Dtype --- - 0 Participant_ID 5789 non-null int64 1 survey_date 4874 non-null datetime64[ns] 2 region 5774 non-null object 3 country 5786 non-null object 4 ip_latitude 5789 non-null float64 5 ip_longitude 5789 non-null float64 6 gender 5789 non-null object 7 age 5789 non-null object 8 height 5789 non-null float64 9 weight 5789 non-null float64 10 bmi 5789 non-null float64 11 blood_type 5689 non-null object 12 insurance 4166 non-null float64 13 income 4636 non-null object 14 smoking 5753 non-null object 15 alcohol 5789 non-null float64 16 cocaine 1430 non-null float64 17 contacts_count 5747 non-null float64 18 public_transport_count 4572 non-null float64 19 working 5761 non-null object 20 worried 4518 non-null float64 21 covid19_positive 5789 non-null int64 22 covid19_symptoms 5789 non-null int64 23 covid19_contact 5789 non-null int64 24 asthma 5789 non-null int64 25 kidney_disease 5789 non-null int64 26 liver_disease 5789 non-null int64 27 compromised_immune 5789 non-null int64 28 heart_disease 5789 non-null int64 29 lung_disease 5789 non-null int64 30 diabetes 5789 non-null int64 31 hiv_positive 5789 non-null int64 32 other_chronic 5789 non-null int64 33 nursing_home 5789 non-null int64 34 health_worker 5789 non-null int64 35 risk_infection 5789 non-null float64 36 risk_infection_level 5789 non-null object 37 risk_mortality 5789 non-null float64 dtypes: datetime64[ns](1), float64(13), int64(15), object(9) memory usage: 1.7+ MB None</pre> Table 1.3. corrected data type

1.3. Variables Requiring Manipulation in Task 3

- **Null values:** Several variables in Table 1.3, including 'survey_date', 'region', 'country', 'blood_type', 'insurance', 'income', 'smoking', 'cocaine', 'contacts_count', 'public_transport_count', 'working', and 'worried', contain missing values. These missing values should be manipulated by data transformation and imputation in Task 3.
- **Categorical variables** need to be converted to binary format through one-hot encoding using 'get_dummies' function for data processing.
- In the data exploration of Task 2, **data issues**, such as outliers, inappropriate values, inconsistent values, and errors, might be discovered. These issues should be resolved through data cleaning and preprocessing in Task 3.
- Highly correlated variables can be utilised to generate derived variables or removed for accuracy. For instance, BMI is closely related to weight and height variables.

TASK 2: Data exploration

Skewness Calculation

	Mean	Median	Mode	Skewness
Participant_ID	164765002.779	60836790.0	1968528	Positive skew
ip_latitude	33.055	39.016	37.816	Unclear / mixed
ip_longitude	-64.964	-81.031	-77.081	Unclear / mixed
height	172.03	172.0	168.0	Positive skew
weight	84.774	82.0	74.0	Positive skew
bmi	28.57	26.8	27.7	Unclear / mixed
insurance	0.851	1.0	1.0	Unclear / mixed
alcohol	1.67	0.0	-1.0	Positive skew
cocaine	-0.205	-1.0	-1.0	Unclear / mixed
contacts_count	7.828	5.0	21.0	Unclear / mixed
public_transport_count	0.416	0.0	0.0	Unclear / mixed
worried	3.531	4.0	4.0	Unclear / mixed
covid19_positive	0.403	0.0	0.0	Unclear / mixed
covid19_symptoms	0.172	0.0	0.0	Unclear / mixed
covid19_contact	0.171	0.0	0.0	Unclear / mixed
asthma	0.131	0.0	0.0	Unclear / mixed
kidney_disease	0.011	0.0	0.0	Unclear / mixed
liver_disease	0.008	0.0	0.0	Unclear / mixed
compromised_immune	0.06	0.0	0.0	Unclear / mixed
heart_disease	0.031	0.0	0.0	Unclear / mixed
lung_disease	0.023	0.0	0.0	Unclear / mixed
diabetes	0.081	0.0	0.0	Unclear / mixed
hiv_positive	0.004	0.0	0.0	Unclear / mixed
other_chronic	0.065	0.0	0.0	Unclear / mixed
nursing_home	0.004	0.0	0.0	Unclear / mixed
health_worker	0.085	0.0	0.0	Unclear / mixed
risk_infection	50.977	37.0	100.0	Unclear / mixed
risk_mortality	0.562	0.156	0.05	Positive skew

Based on the logic in the Week 9 Tutorial, the skewness of the data was measured using mean, median, and mode values. However, some values do not completely fit the logic, so we will additionally check skewness through the Distplot graph.

The following code is used to first search for all variables and identify missing values, inappropriate/inconsistent values and errors. The total number of data should be 5789, and if the number of data doesn't fill this number, this suggests that there are missing values.

```
for col in df.columns:
    print(f"\n===== {col} =====")

    print(df[col].describe())
    print("\n -----")
    print(df[col].value_counts().head(10))
    print("\n -----")
    print(df[col].unique())
```

For numerical data, the data were visualised using boxplots and distplot logic. The boxplot graphs allow for easy visualisation of data and the presence of outliers. Additionally, distplot complements the more detailed visualisation of data that the boxplot could not provide by checking the frequency, skewness and distribution of the data.

```
# outlier
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

columns_to_plot = [
    'risk_mortality', 'cocaine', 'public_transport_count',
    'weight', 'contacts_count', 'height', 'ip_latitude', 'ip_longitude'
]

for col in columns_to_plot:
    # Boxplot
    ax = sns.boxplot(df[col])
    plt.title(f'Boxplot of {col}')
    plt.show()

    # Distplot
    dg = sns.distplot(df[col])
    plt.title(f'Distribution of {col}')
    plt.show()
```

In the case of categorical variables, data were visualised using the count plot code. Although skewness cannot be measured, data for each category were visualised in a bar

graph to confirm data bias. After that, dummy processing is planned for one-hot encoding, which is essential for overall data comparison.

```
import seaborn as sns
import matplotlib.pyplot as plt

cols = ['region', 'gender', 'age', 'blood_type',
        'insurance', 'smoking', 'working',
        'worried', 'covid19_positive', 'covid19_symptoms',
        'covid19_contact', 'asthma', 'kidney_disease', 'liver_disease',
        'compromised Immune', 'heart_disease', 'lung_disease', 'diabetes',
        'hiv_positive', 'other_chronic', 'nursing_home', 'health_worker', 'risk_infection_level']

for col in cols:
    plt.figure(figsize=(7,4))
    sns.countplot(x=col, data=df)
    plt.title(f'Countplot of {col}')
    plt.show()
```

```
<DatetimeArray>
['2020-02-12 00:00:00', '2020-02-07 00:00:00', '2020-05-07 00:00:00',
 'NAT', '2020-04-07 00:00:00', '2020-07-07 00:00:00',
 '2020-06-04 00:00:00', '2020-04-06 00:00:00', '2020-06-07 00:00:00',
 '2020-03-07 00:00:00', '2020-06-12 00:00:00', '2020-01-07 00:00:00',
 '2020-08-07 00:00:00', '2020-07-12 00:00:00', '2020-03-12 00:00:00',
 '2020-09-12 00:00:00', '2020-04-12 00:00:00', '2020-03-06 00:00:00',
 '2020-01-04 00:00:00', '2020-05-04 00:00:00', '2020-08-04 00:00:00',
 '2020-04-05 00:00:00', '2020-05-12 00:00:00', '2020-08-12 00:00:00',
 '2020-12-12 00:00:00', '2020-10-12 00:00:00', '2020-02-04 00:00:00',
 '2020-03-04 00:00:00', '2020-04-04 00:00:00', '2020-07-04 00:00:00',
 '2020-09-04 00:00:00', '2020-10-04 00:00:00', '2020-11-04 00:00:00',
 '2020-12-04 00:00:00', '2020-06-05 00:00:00', '2020-06-06 00:00:00',
 '2020-10-06 00:00:00', '2020-08-06 00:00:00', '2020-12-05 00:00:00',
 '2020-01-06 00:00:00', '2020-07-06 00:00:00', '2020-12-06 00:00:00',
 '2020-06-05 00:00:00', '2020-07-05 00:00:00', '2020-02-06 00:00:00',
 '2020-05-06 00:00:00', '2020-09-05 00:00:00', '2020-11-12 00:00:00',
 '2020-09-06 00:00:00', '2020-10-05 00:00:00', '2020-11-06 00:00:00',
 '2020-01-05 00:00:00', '2020-05-05 00:00:00', '2020-03-05 00:00:00',
 '2020-11-05 00:00:00']
Length: 55, dtype: datetime64[ns]
```

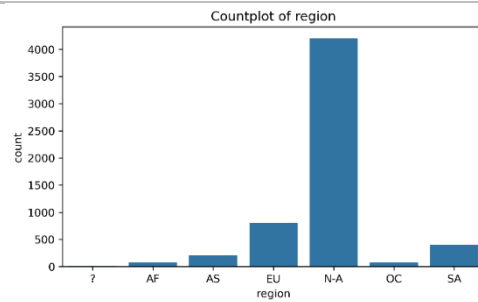
1. Data Name: Survey_date

There is a total of 915 missing values, the date range matches the COVID-19 period, and no outliers or errors are found. Task 1 has already been converted to Object to Datetime, and further transformations consider replacing the missing values with other values or removing them if no variable analysis is required.

```
=====region=====
count    5774
unique      7
top        N-A
freq      4203
Name: region, dtype: object

-----
region
N-A    4203
EU      801
SA      396
AS      206
AF       79
OC       77
?         12
Name: count, dtype: int64

-----
['?', 'AF', 'AS', 'EU', nan, 'N-A', 'OC', 'SA']
```



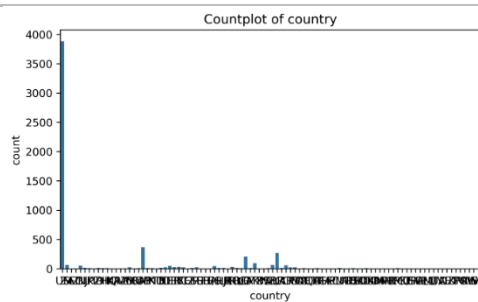
2. Data Name: Region

Unknown '?' data and a total of 15 missing values were found. Approximately 72.8% are N-A, which is overly biased towards the N-A region. This ratio shows that the data are unbalanced and of low quality, so removing variables can be considered.

```
=====country=====
count    5786
unique    96
top        US
freq      3885
Name: country, dtype: object

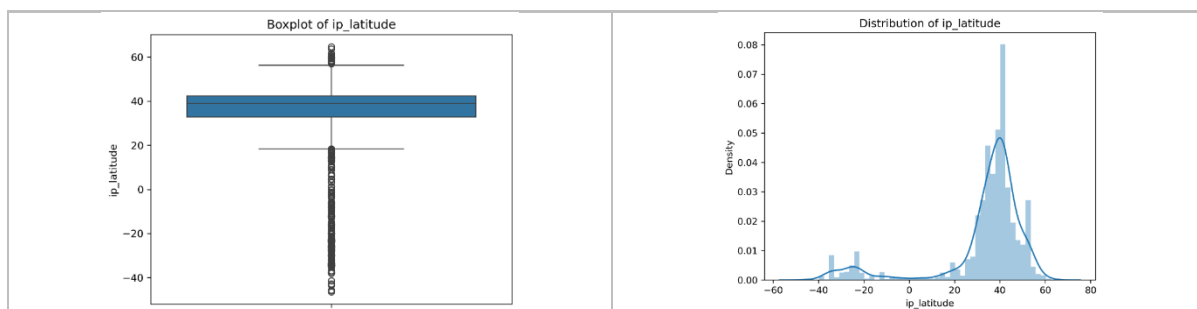
-----
country
US    3885
GB     355
BR     270
CA     208
PK      94
AU      68
ZA      65
AR      60
IN      54
DE      48
Name: count, dtype: int64

-----
['US', 'ZA', 'NZ', 'EG', 'DN', 'EL', 'JP', 'KZ', 'DD', 'PH', 'HK', 'IQ', 'IR', 'AZ',
 'MD', 'PK', 'HY', 'SA', 'GB', 'PK', 'PT', 'MD', 'IE', 'NL', 'DE', 'FR', 'IT', 'BE',
 'CZ', 'GB', 'SE', 'SI', 'FI', 'BA', 'ES', 'MD', 'HR', 'DE', 'MD', 'RU', 'BG', 'CA',
 'GT', 'MX', 'KY', 'BS', 'NZ', 'AU', 'BR', 'CL', 'AR', 'CO', 'PE', 'NA', 'AE', 'BD',
 'QA', 'IR', 'BH', 'GE', 'AT', 'PL', 'CH', 'MD', 'AD', 'RS', 'SK', 'MT', 'DK', 'DO',
 'BM', 'PA', 'HN', 'PR', 'CB', 'EC', 'AN', 'MG', 'KE', 'TW', 'SG', 'NP', 'MY', 'LU', 'LT',
 'UA', 'VU', 'DZ', 'ET', 'KR', 'PS', 'RM', 'AH', 'LV', 'SV']
```



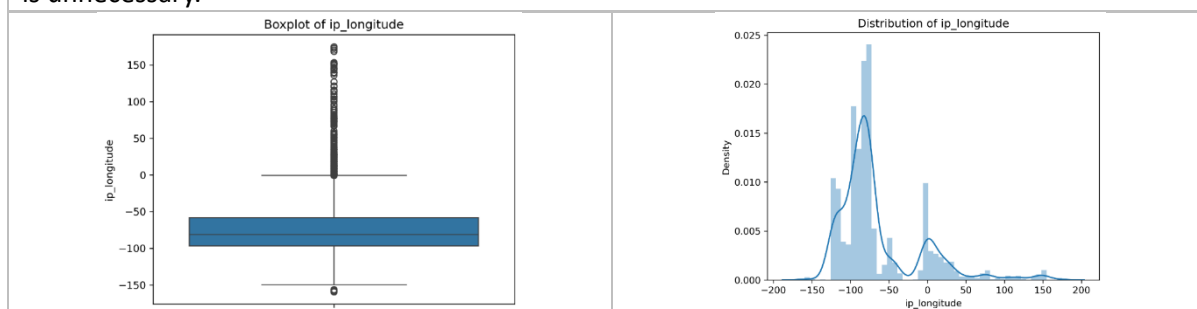
3. Data Name: Country

There are three missing values, and the data is concentrated in certain countries; there are several countries with fewer than 10 samples. Since approximately 67% of the data are biased to the US, it is possible to consider how to substitute missing values with the US and the possibility of separately categorising countries with few samples.



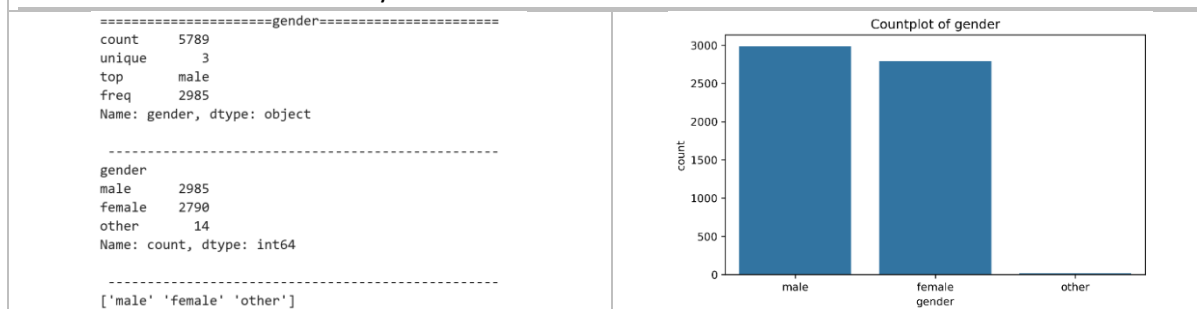
4. Data Name: `ip_latitude`

The number of data points is 5789, so there are no missing values, and all data are within the valid range (-90 to +90). Although outliers are measured in the boxplot, it is difficult to define them as outliers because they are values that describe geographic locations, and it can be confirmed that the sample is dense in the northern hemisphere (Negative skewness). Subsequent data conversion is unnecessary.



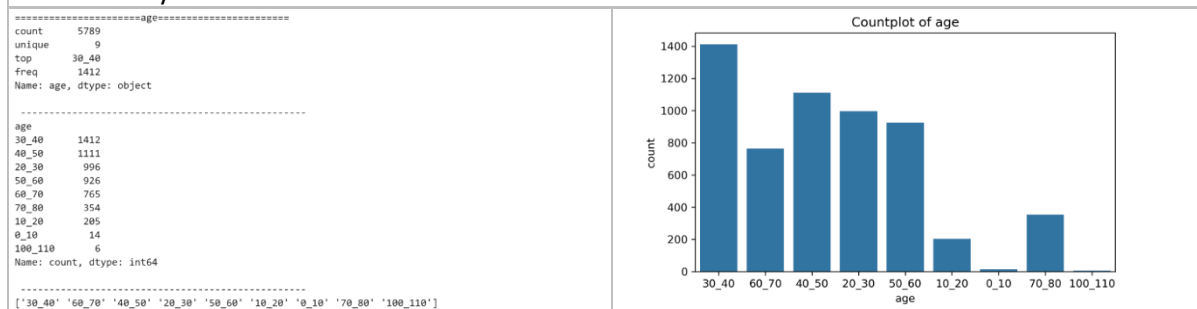
5. Data Name: `ip_longitude`

Same as Latitude, the number of data is 5789, no missing values and all data are within the valid range (-180 to +180). Since it is a value that describes a geographic location, it is difficult to define outliers, and it can be confirmed that the sample is concentrated in a specific area. Subsequent data conversion is unnecessary.



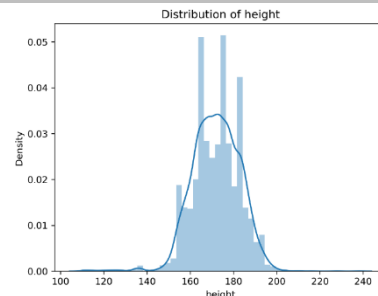
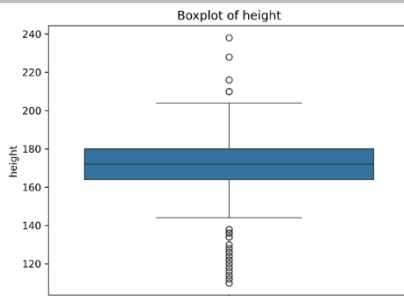
6. Data Name: `Gender`

No missing values and outliers/errors. The distribution between genders is similar. In the case of the 'other' category, the number of samples is as small as 14, so subsequent transformation seems unnecessary.



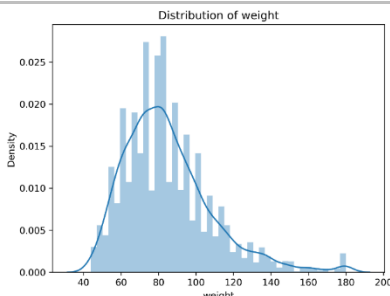
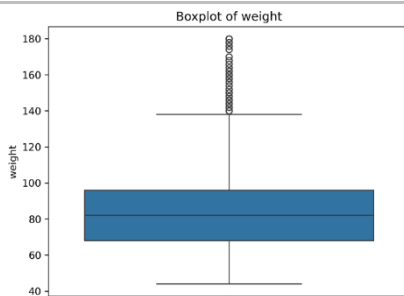
7. Data Name: Age

There are no missing values, outliers, errors and extra data conversion seems unnecessary. Data which excludes extremely young or old (10s and 100s) is also relatively evenly distributed (symmetric) and identified as meaningful data.



8. Data Name: Height

There were no missing values, outliers were measured partially, and the data were evenly distributed. Considering the age group, low height is a range to consider, but high height of 210 cm or more is a very rare case. However, since comprehensive judgment is possible through BMI variables together with the weight variable, additional data conversion is unnecessary.



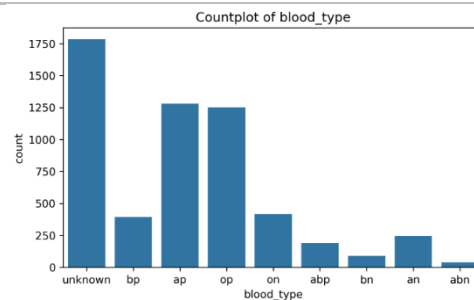
9. Data Name: Weight

There are no missing values, and the data show that the shape of the tail is relatively evenly positive skewness. Like Height, the weight of the 140kg costume is not in the normal range, but since comprehensive judgment is possible through BMI variables, additional data conversion is also unnecessary for Weight.

```
=====blood_type=====
count      5689
unique         9
top      unknown
freq       1784
Name: blood_type, dtype: object

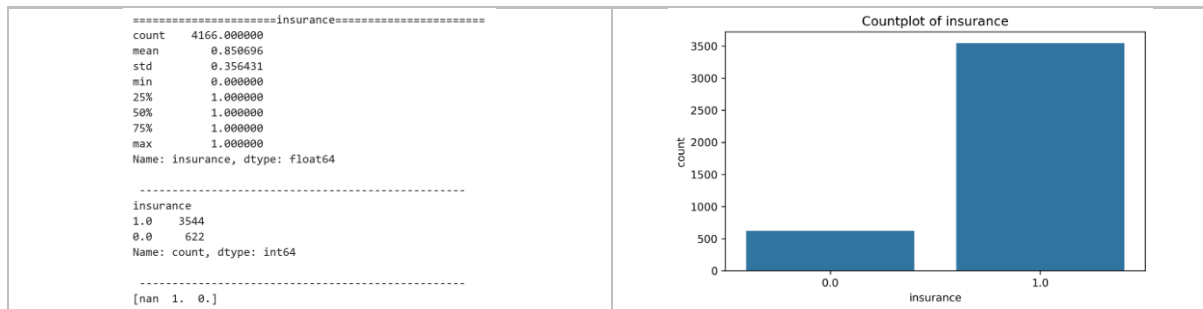
-----
blood_type
unknown    1784
ap         1279
op         1251
on         417
bp         394
an         244
abp        191
bn          91
abn         38
Name: count, dtype: int64

-----
['unknown' 'bp' 'ap' 'op' 'on' 'abp' 'bn' 'an' 'abn' nan]
```



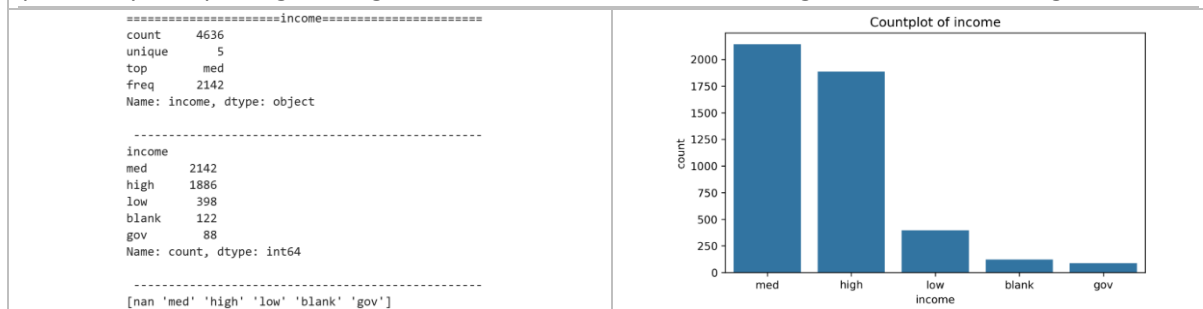
10. Data Name: Blood Type

You can find 100 missing values and the 'Unknown' category. In fact, Unknown has the same meaning as missing values, so if they are replaced with missing values, they account for about 32.5% of the total data. If this is replaced with the mode value of the AP variable, it becomes unbalanced and unreliable data at the same time. Therefore, you can maintain the Unknown category in which missing values are replaced or consider removing variables later.



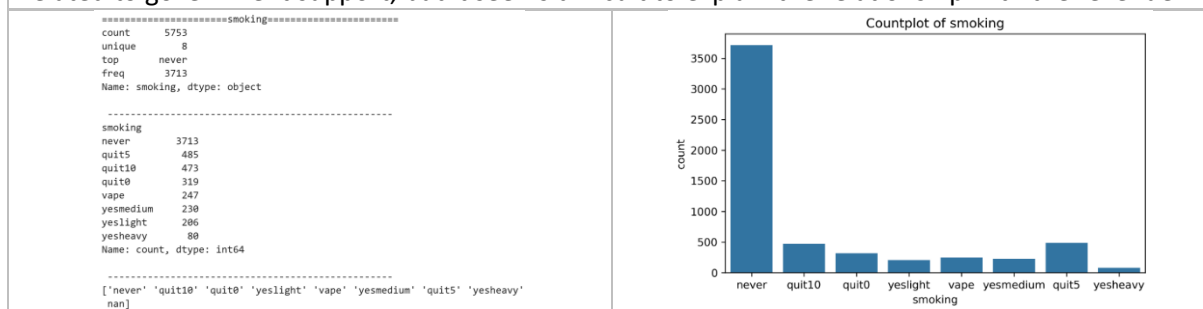
11. Data Name: Insurance

In Task 1, the blank value was treated as Nan. There are 1623 missing values, which is about 28% of the total data. Since the data is biased to 'have insurance', it is possible to consider the possibility of replacing missing values with the mode or converting them into new categories.



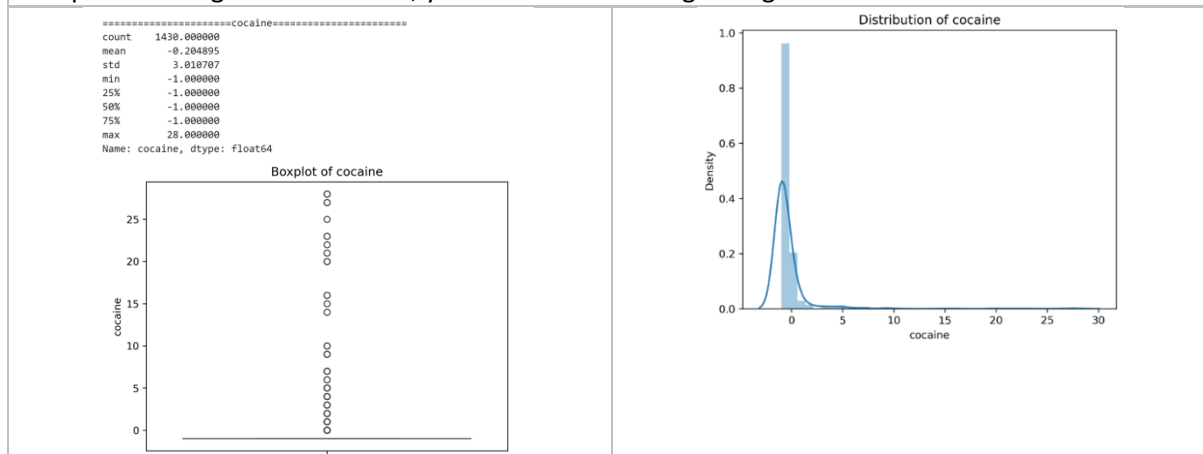
12. Data Name: Income

Missing values are 1153, and 'blank' values are 122. The 'government' category seems to be related to government support, but it seems difficult to explain the relationship with the revenue.



13. Data Name: Smoking

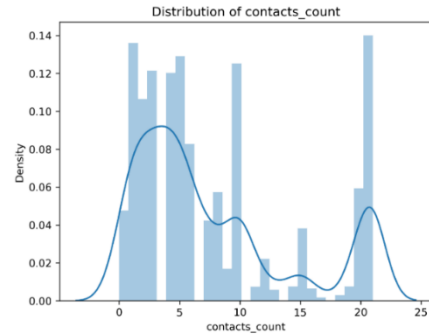
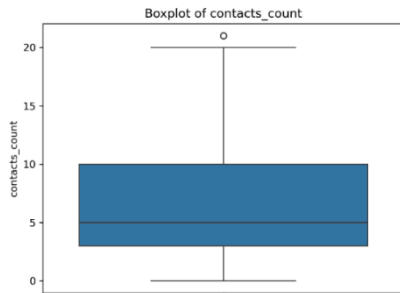
There are 36 missing values, and outliers and error values do not exist, but the distinction between samples is ambiguous. After that, you can consider categorising clearer criteria in Task 3.



14. Data Name: Cocaine

There are 4359 missing values, which is about 75.3% of all samples. It shows extreme positive skewness, and most of the values are concentrated at 0, so all non-zero data are displayed as outliers. It is judged that the accuracy and value of the data are low, and subsequent removal of variables can be considered.

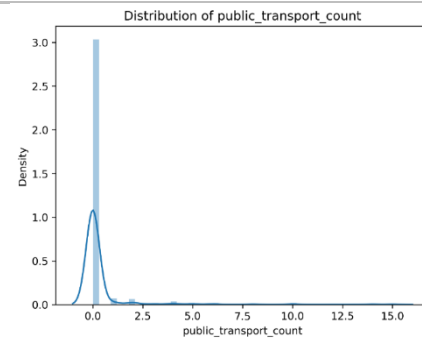
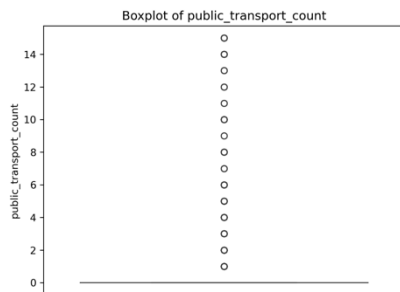
```
=====contacts_count=====
count    5747.000000
mean      7.828432
std       6.672943
min       0.000000
25%       3.000000
50%       5.000000
75%      10.000000
max       21.000000
Name: contacts_count, dtype: float64
```



15. Data Name: Contact_count

There are 42 missing values. The mean value is greater than the median value, so positive skewness can be observed, and the standard deviation of 6.67 shows a large difference in the number of contacts between individuals.

```
=====public_transport_count=====
count    4572.000000
mean      0.415573
std       1.811049
min       0.000000
25%       0.000000
50%       0.000000
75%       0.000000
max       15.000000
Name: public_transport_count, dtype: float64
```

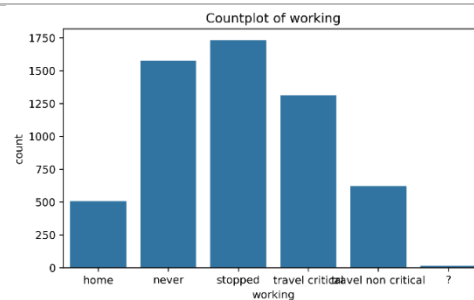


16. Data name: Number of public transportation

There are 1217 missing values, and 4162 samples are concentrated at zero. It shows strong positive skewness and seems to be related to the decrease in public transportation use during COVID-19. After that, the possibility of creating a variable for total contact, along with the Contacts_count variable, can be considered.

```
=====working=====
count    5761
unique    6
top      stopped
freq     1732
Name: working, dtype: object
```

```
-----
working
stopped    1732
never      1577
travel critical  1312
travel non critical  620
home        506
?           14
Name: count, dtype: int64
-----
['home' 'never' 'stopped' 'travel critical' 'travel non critical' '?' nan]
```



17. Data Name: Working

Twenty-eight missing values and an unknown category '?' were also identified. Other than that, it shows a relatively balanced distribution. After that, you can consider the possibility of replacing

missing values in the mode or making it a new category with '?'

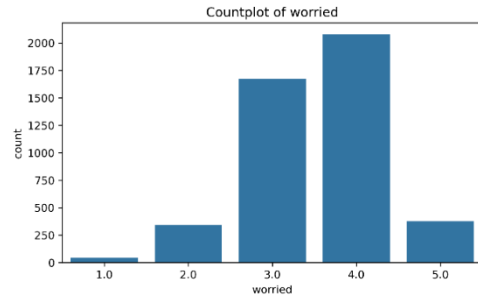
```

=====worried=====
count    4518.000000
mean      3.538987
std       0.792812
min       1.000000
25%       3.000000
50%       4.000000
75%       4.000000
max       5.000000
Name: worried, dtype: float64

-----
worried
4.0    2079
3.0    1674
5.0     377
2.0     342
1.0      46
Name: count, dtype: int64

-----
[ 3.  2.  4. nan  5.  1.]

```



18. Data Name: Worried

1271 missing values were observed, and all samples were within the valid range. It can be seen that most people have a high level of worry, and after that, a method of replacing missing values with the mode, mean or creating a new category can be considered.

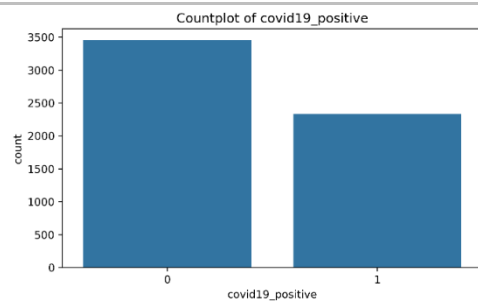
```

=====Covid19_positive=====
count    5789.000000
mean      0.403351
std       0.490612
min       0.000000
25%       0.000000
50%       0.000000
75%       1.000000
max       1.000000
Name: covid19_positive, dtype: float64

-----
covid19_positive
0     3454
1     2335
Name: count, dtype: int64

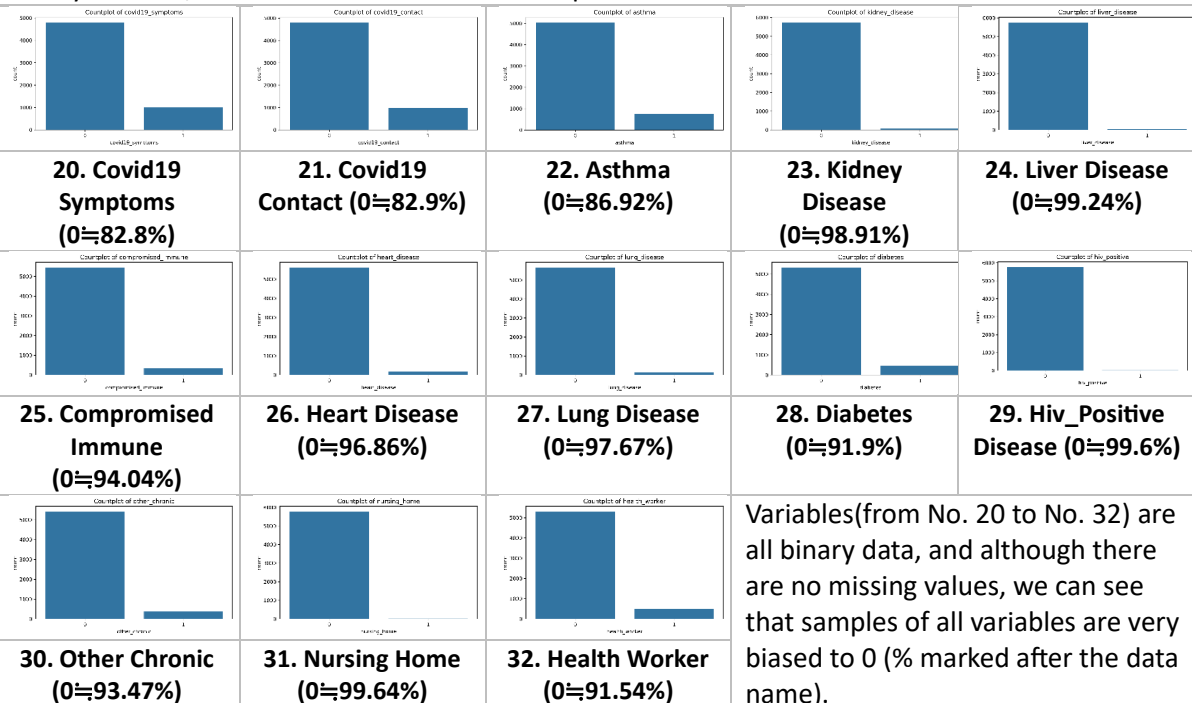
-----
[1 0]

```

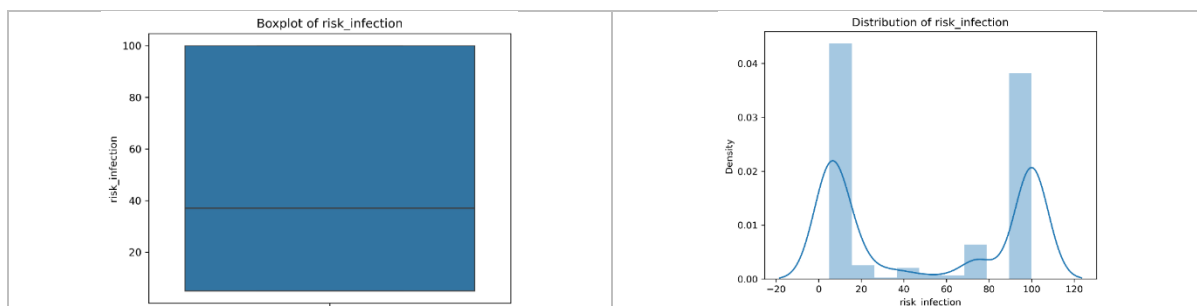


19. Covid19_positive

There were no missing values, and samples with an evenly distributed ratio of COVID-19 positive (about 59.7%) and negative (about 40.3%) appeared. This is very significant data, and since it is a binary variable, no further transformation is required.

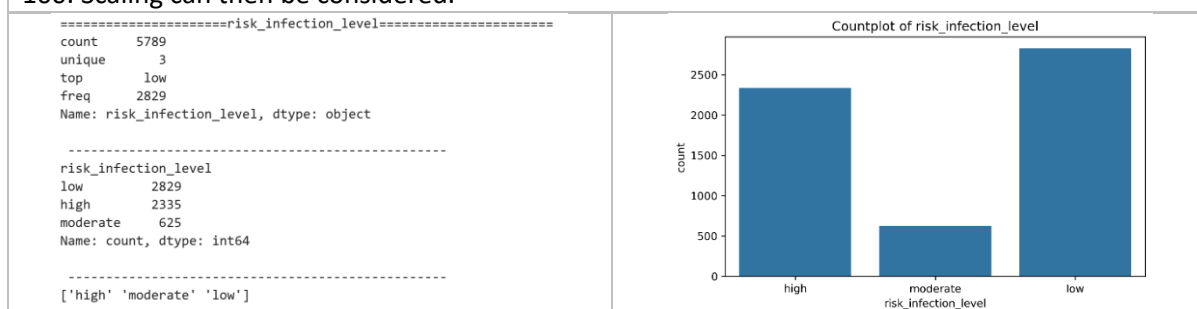


20-32. No additional transformation is required, but a method of simplifying variables by grouping underlying diseases into one can be considered.



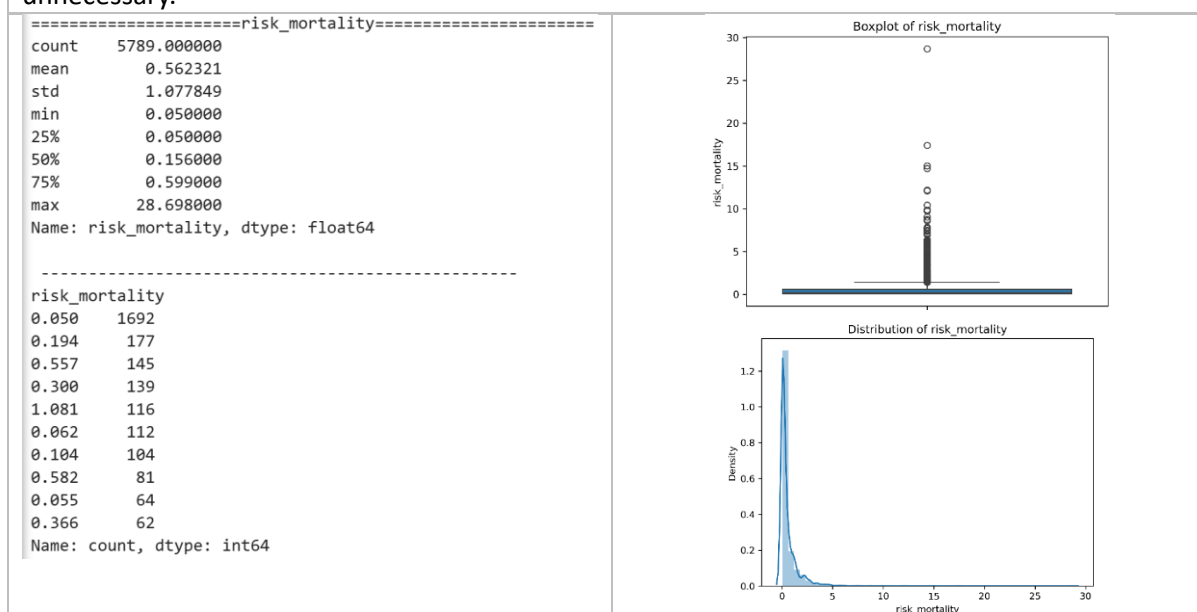
33. Data Name: Risk Infection

There are no missing values, outliers, or errors, and samples are variously distributed from 0 to 100. Scaling can then be considered.



34. Data Name: Risk infection level

It is a variable divided by the level of 'risk infection'. There are no missing values, and it can be seen that it is slightly biased toward low. It is high-quality data, and subsequent data conversion is unnecessary.



35. Data Name: Risk Mortality

There is no missing value, but it shows a strong positive skewness. Within this data, it means that most people have a very low risk of death, meaning that the fatality rate of COVID-19 is low. About 29% of the data have a 0.05 value, and most have a lower value than 0.599. However, observing the figure of maximum and the graphs shows that a small number of samples have extreme values. The use of these extreme data can distort the model, which requires further consideration of transformation or removal.

Answer the following questions

(a). What is the average contacts_count of a patient from the US who is COVID-19 positive? **8.244**

```
# (a) the average contacts_count of a patient from the US who is Covid-19 positive
avg_contacts_us = df[(df['country']=='US') & (df['covid19_positive']==1)]['contacts_count'].mean()
print("Average contacts_count of US COVID-19 positive patients: ", avg_contacts_us)
```

Average contacts_count of US COVID-19 positive patients: 8.24438902743142

(b). Which country has the highest number of covid19_positive? **US (1627)**

```
# (b) which country has the highest number of 'Covid19-positive'
positive_number = df[df['covid19_positive']==1].groupby(['country'])['covid19_positive'].value_counts()
print(positive_number.sort_values(ascending=False))
```

country	covid19_positive	
US	1	1627
BR	1	164
GB	1	132
CA	1	61
MX	1	40

(c). How many health workers in the dataset? and how many of them are covid19 positive?

The number of Health worker: **490**, The number of Health worker(covid19 positive): **182**

```
# (c) How many health workers in the dataset? and how many of them are covid19 positive?
print(df.groupby(['health_worker'])['covid19_positive'].value_counts())
```

health_worker	covid19_positive	
0	0	3146
	1	2153
1	0	308
	1	182

Name: count, dtype: int64

(d). Which are the top-five countries according to the average mortality risk?

AD, AE, AM, AR, AT

```
# (d) Which are the top-five countries according to the average mortality risk?
mortality_mean = df.groupby(['country'])['risk_mortality'].mean()
print(mortality_mean.head(5))
```

country	
AD	3.9940
AE	0.2391
AM	1.9235
AR	0.2470
AT	0.3556

Name: risk_mortality, dtype: float64

TASK 3: Data Preparation

3.1. Summary of data exploration

Through the data exploration, several data quality issues were discovered in the dataset:

- **Missing data & Incorrect values:** Null values of all variables can be calculated by using the Non-Null count row of 'info()' function. Additionally, 'get(0)' and 'mask' functions were used to find errors and invalid values.
 - Missing values: null value and 'unknown' value (e.g. 'survey_date', 'region', 'blood_type', 'insurance', 'income', 'cocaine', 'contacts_count', 'working', 'worried')
 - Incorrect values: '?', 'blank' values (e.g. 'region', 'insurance', 'income', 'working')
- **Outlier:** Outliers can be found in numerical variables by the boxplot.
 - Outliers exist in 'risk_mortality', 'cocaine', 'public_transport_count', 'weight', 'contacts_count', 'height', 'ip_latitude', 'ip_longitude'
 - 'cocaine' and 'transport_count' variables have a lot of null values, so it is not

appropriate to remove outliers.

- **Categorical variables:** categorical variables will be converted to numerical values by utilising 'get_dummies' function called by one-hot encoding. Furthermore, count plots of categorical variables show values to be skewed to one side. For instance, the values of most data about 'region' are 'N-A'.
- There are **several strong skewnesses** in the dataset, which are 'risk_mortality', 'smoking', and 'contact_count'. The existence of skewness variables was confirmed by comparing the mean, median, and mode values of each variable.

3.2. Data preparation

- **Data cleaning:** the imputation of values to enhance data quality
 - Missing data: ignore, replace, use a global constant, replace the attribute mean or mode.
- **Data transformation:** convert data to the correct format to apply machine learning models.
 - **Data type:** 'pd.to_datetime' and 'astype' codes changed data types to the corrected data types of variables in Task 1.
 - **Categorical values** should be converted to binary and Boolean values for visualisation and decision tree models using 'get_dummies()' function.
 - **Normalisation:** the process that scales the data into a fixed range, including between 0 and 1. It is effective to compare the values of different features. This report will utilise 'MinMaxScaler' to scale data.

3.3. The process of data preparation: Python code and output

This project used 'unique()' and 'value_counts()' functions to confirm missing and invalid values in Task2. IQR(interquartile range) was utilised to determine the standard to remove outliers.

3.3.1. Missing and incorrect values

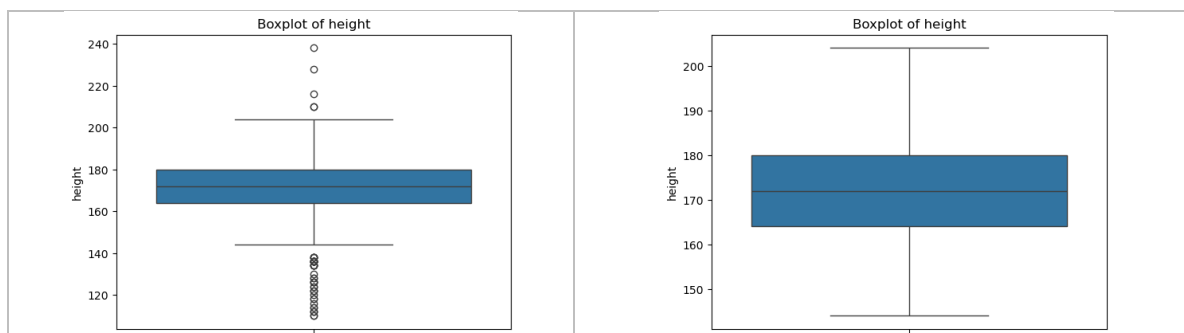
Python code snippets & outputs	
survey_date	region
▫ Missing value: NaT <pre># Survey_date: NaT df['survey_date'] = df['survey_date'].fillna(df['survey_date'].mode()[0])</pre> 'NaT' was changed to the mode value of 'survey_date'. The most appropriate value to change is the mode value. If the mean value of 'survey_value' is utilised, the value of 'NaT' can have time value, including hour, minute, and second. These values regarding time are unnecessary. <pre>=====survey_date===== DatetimeIndex: [2020-02-12 00:00:00, 2020-02-07 00:00:00, 2020-05-07 00:00:00, 2020-06-07 00:00:00, 2020-04-07 00:00:00, 2020-07-07 00:00:00, 2020-06-04 00:00:00, 2020-04-06 00:00:00, 2020-03-07 00:00:00, 2020-06-12 00:00:00, 2020-01-07 00:00:00, 2020-08-07 00:00:00, 2020-07-12 00:00:00, 2020-03-12 00:00:00, 2020-09-12 00:00:00, 2020-04-12 00:00:00, 2020-03-06 00:00:00, 2020-01-04 00:00:00, 2020-05-04 00:00:00, 2020-08-04 00:00:00, 2020-04-05 00:00:00, 2020-09-12 00:00:00, 2020-09-12 00:00:00, 2020-12-12 00:00:00, 2020-10-12 00:00:00, 2020-02-04 00:00:00, 2020-03-04 00:00:00, 2020-04-04 00:00:00, 2020-07-04 00:00:00, 2020-09-04 00:00:00, 2020-10-04 00:00:00, 2020-11-04 00:00:00, 2020-12-04 00:00:00, 2020-06-05 00:00:00, 2020-06-06 00:00:00, 2020-10-06 00:00:00, 2020-08-06 00:00:00, 2020-12-06 00:00:00, 2020-01-06 00:00:00, 2020-07-06 00:00:00, 2020-12-06 00:00:00, 2020-06-05 00:00:00, 2020-07-06 00:00:00, 2020-02-06 00:00:00, 2020-05-06 00:00:00, 2020-09-06 00:00:00, 2020-11-12 00:00:00, 2020-09-06 00:00:00, 2020-10-05 00:00:00, 2020-11-06 00:00:00, 2020-01-05 00:00:00, 2020-01-05 00:00:00, 2020-03-05 00:00:00, 2020-11-05 00:00:00] Length: 54, dtype: datetime64[ns]</pre>	▫ Missing value: nan ▫ Invalid value: '?' <pre># region: ?, NaN df['region'] = df['region'].replace('?', np.nan) df['region'] = df['region'].fillna(df['region'].mode()[0])</pre> '?' was converted to a nan value. After that, NaN (missing value) was changed mode value of the 'region'. It is proper for categorical values to change mode or median value. In this variable, missing values also have a possibility to be 'N-A' value, in that most values are 'N-A'. <pre>=====region===== ['N-A' 'AF' 'AS' 'EU' 'OC' 'SA']</pre>

income - Missing value: nan - Invalid value: 'blank' <pre># income df['income']=df['income'].replace('blank', np.nan) df['income'] = df['income'].fillna('unknown')</pre> Firstly, 'blank' was changed to nan because 'blank' means missing value. Secondly, nan values were changed to 'unknown' values. That is why the nan values were in 1275 cases. For this reason, it was more appropriate to create a new variable named 'unknown' rather than to replace other values. <pre>=====income===== ['unknown' 'med' 'high' 'low' 'gov']</pre>	country <pre><class 'pandas.core.frame.DataFrame'> RangeIndex: 5789 entries, 0 to 5788 Data columns (total 38 columns): # Column Non-Null Count Dtype --- - 0 Participant_ID 5789 non-null int64 1 survey_date 5789 non-null object 2 region 5774 non-null object 3 country 5786 non-null object</pre> Missing value: 5789(total entries) – 5786(Non-null count of country) = 3 missing values. <pre># country df['country'] = df['country'].fillna(df['country'].mode()[0])</pre> The missing value was changed to the mode value because most values in the 'country' variable are 'US'. <pre>=====country===== ['US' 'ZA' 'MZ' 'EG' 'IN' 'IL' 'JP' 'KZ' 'ID' 'PH' 'HK' 'IQ' 'IR' 'AZ' 'MO' 'PK' 'MY' 'SA' 'GB' 'MK' 'PT' 'NO' 'IE' 'NL' 'DE' 'FR' 'IT' 'BE' 'CZ' 'GR' 'SE' 'SI' 'FI' 'BA' 'ES' 'HU' 'HR' 'JE' 'RO' 'RU' 'BG' 'CA' 'GT' 'MX' 'KY' 'BS' 'NZ' 'AU' 'BR' 'CL' 'AR' 'CO' 'PE' 'MA' 'AE' 'BD' 'QA' 'TR' 'BH' 'GE' 'AT' 'PL' 'CH' 'MD' 'AD' 'RS' 'SK' 'MT' 'DK' 'DO' 'BM' 'PA' 'HN' 'PR' 'CR' 'EC' 'MU' 'KE' 'TH' 'SG' 'NP' 'AM' 'LU' 'LT' 'UA' 'VI' 'DZ' 'ET' 'KR' 'PS' 'MM' 'AN' 'LV' 'SV']</pre>
blood_type Missing value: nan <pre># blood type df['blood_type'] = df['blood_type'].fillna('unknown')</pre> nan value was changed to 'unknown'. 'unknown' values cannot be ignored because 1784 cases have 'unknown' value. Therefore, a missing value was included in 'unknown' value. <pre>=====blood_type===== ['unknown' 'bp' 'ap' 'op' 'on' 'abp' 'bn' 'an' 'abn']</pre>	cocaine Missing value: nan <pre># cocaine: nan df['cocaine']=df['cocaine'].fillna(df['cocaine'].median())</pre> The average value can be distorted because there are 1027 cases with -1.0 value and 4 cases with 28.0 value. Therefore, the median value is more stable. <pre>=====cocaine===== [0. 1. -1. 15. 3. 7. 5. 2. 9. 4. 10. 6. 20. 14. 23. 21. 28. 22. 25. 27. 16.]</pre>
contact_count Missing value: nan <pre>#contacts_count:nan df['contacts_count']=df['contacts_count'].fillna(df['contacts_count'].median())</pre> Most cases have from 1 to 6 values, whereas 626 cases have 21.0 values. Therefore, the median value is appropriate to impute the variable. <pre>=====contacts_count===== [10. 6. 5. 4. 3. 16. 15. 7. 2. 20. 21. 1. 8. 12. 9. 14. 11. 13. 19. 18. 17.]</pre>	public_transport_count Missing value: nan <pre>#public_transport_count:nan df['public_transport_count']=df['public_transport_count'].fillna(df['public_transport_count'].median())</pre> Most value is concentrated at zero. Therefore, the mean value is sensitive to outlier, and the mode value is proper. <pre>=====public_transport_count===== [0. 2. 4. 6. 3. 1. 12. 5. 10. 8. 14. 9. 15. 11. 7. 13.]</pre>
working - Missing value: nan - Invalid value: '?' <pre># working: ?, NaN df['working'] = df['working'].replace('?', np.nan) df['working'] = df['working'].fillna(df['working'].mode()[0])</pre> '?' value was changed to a NaN value. The missing value was converted to the mode value of the overall 'working' values because most people had similar working situations in the COVID-19 pandemic. <pre>=====working===== ['home' 'never' 'stopped' 'travel critical' 'travel non critical']</pre>	worried Missing value: nan <pre># worried: NaN df['worried'] = df['worried'].fillna(df['worried'].median())</pre> 'worried' variable is an ordinal variable regarding the extent of worry about COVID-19. Therefore, the median value can represent the centre value better than the mean value. <pre>=====worried===== [3. 2. 4. 5. 1.]</pre>

country	The results of the non-NaN count
□ Missing value: nan <pre># insurance df['insurance'] = df['insurance'].fillna(df['insurance'].mode()[0])</pre> It is reasonable for missing values to convert to the mode values of the 'insurance' variable because of the binary variable. <pre>=====insurance===== [1. 0.]</pre>	<pre>missing_value = ['survey_date', 'region', 'worried', 'working', 'smoking', 'cocaine', 'contacts_count', 'public_transport_count', 'blood_type', 'insurance', 'income', 'country'] for col in missing_value: print(f" After filling out the missing values of < {col} >, non-NaN count= ", df[col].count())</pre> After filling out the missing values of < survey_date >, non-NaN count= 5789 After filling out the missing values of < region >, non-NaN count= 5789 After filling out the missing values of < worried >, non-NaN count= 5789 After filling out the missing values of < working >, non-NaN count= 5789 After filling out the missing values of < smoking >, non-NaN count= 5789 After filling out the missing values of < cocaine >, non-NaN count= 5789 After filling out the missing values of < contacts_count >, non-NaN count= 5789 After filling out the missing values of < public_transport_count >, non-NaN count= 5789 After filling out the missing values of < blood_type >, non-NaN count= 5789 After filling out the missing values of < insurance >, non-NaN count= 5789 After filling out the missing values of < income >, non-NaN count= 5789 After filling out the missing values of < country >, non-NaN count= 5789 The proper processing of missing values can be confirmed by using print() and count() functions. According to these outputs, the missing values have all been resolved. (Total entries: 5789)

3.3.2. Outlier ('risk_mortality', 'weight', 'height')

<pre>outlier_cols = ['risk_mortality', 'weight', 'height'] # cocaine column have a lot of blank so outlier of this column cannot be removed for col in outlier_cols: Q1 = df[col].quantile(0.25) Q3 = df[col].quantile(0.75) IQR = Q3 - Q1 # conditional-outlier lower_bound = Q1 - 1.5 * IQR upper_bound = Q3 + 1.5 * IQR # imputation: median df.loc[(df[col] < lower_bound) (df[col] > upper_bound), col] = np.nan df[col] = df[col].fillna(df[col].median())</pre>	Outliers can be measured in numerical variables using the quartile method: - Q1 (value at 25%), Q3 (value at 75%) - IQR (interquartile range): Q3 – Q1 - Outliers were defined using IQR standards, such as 1.5 * IQR from Q1 and Q2. Furthermore, the outliers were converted to NaN values, which were then replaced with the median value.
Before	After
risk_mortality	
weight	
height	



- Outliers of 'risk_mortality', 'weight' and 'height' variables were processed by converting outliers to the median value based on IQR.
- 'cocaine', 'public_transport_count' have a lot of null values, so if outliers are removed, most values can change to the median value. This situation can lead to misleading data.
- Additionally, 'public_transport_count' and 'contacts_count' will be used to make a derived variable, focusing on the sum of the number of contact people. That is why outliers of these variables were not handled in this part. Therefore, the exploration of outliers will be processed after creating derived variables.

3.4. Derived variables

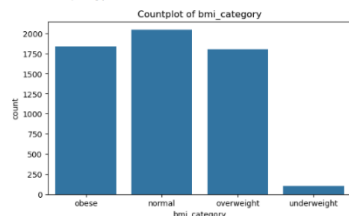
bmi_category: - Categorises bmi values into 'underweight', 'normal', 'overweight', and 'obese' - The extent of the BMI was classified for intuitive recognition, since bmi values are continuous floating numbers. <pre># bmi_category: underweight, normal, overweight def bmi_category(bmi): if bmi < 18.5: return 'underweight' elif bmi < 25: return 'normal' elif bmi < 30: return 'overweight' else: return 'obese' df['bmi_category'] = df['bmi'].apply(bmi_category)</pre>	smoking_category - Classifies smoking variable with 'current', 'former', and 'never'. 'smoking' variable has complicated values, such as 'never', 'quit10', 'quit0', 'yeslight', 'vape', 'yesmedium', 'quit5', 'yesheavy'. This situation can lead to an increase in the complexity of data. <pre># smoking_group def smoking_category(x): if x in ['yeslight', 'vape', 'yesmedium', 'yesheavy']: return 'current' elif x in ['quit0', 'quit10', 'quits']: return 'former' elif x in ['never']: return 'never' else: return 'unknown' df['smoking_group'] = df['smoking'].apply(smoking_category)</pre>
exposure_count - The sum of 'contacts_count' and 'public_transport_count' - The important factor is how many people were contacted for the prediction of COVID-19. 'public_transport_count' can also not be meaningful because it has many null values. <pre># exposure_count (contact count + transport count) df['exposure_count'] = df['contacts_count'] + df['public_transport_count']</pre>	Underlying_disease - Counts the number of chronic diseases of each person, using variables related to disease, such as 'asthma', 'kidney_disease', 'liver_disease' and 'heart_disease'. - That is why there is not enough data regarding chronic diseases. For instance, 'liver_disease' variable has 5745 cases with a value of 0 and only 44 cases with 1 value. Furthermore, the sum of having diseases can become meaningful data, improving the relationship with COVID-19. <pre># comorbidity chronic_cols = ['asthma', 'kidney_disease', 'liver_disease', 'compromised Immune', 'heart_disease', 'lung_disease', 'diabetes', 'hiv_positive', 'other_chronic'] df['underlying_disease'] = df[chronic_cols].sum(axis=1)</pre>

- The data quality of derived variables

bmi_category	smoking_group
--------------	---------------

```
=====bmi_category=====
['obese' 'normal' 'overweight' 'underweight']

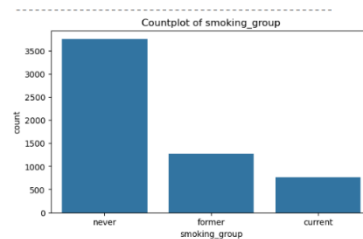
-----
bmi_category
normal      2045
obese       1839
overweight  1806
underweight   99
Name: count, dtype: int64
```



The count plot was evenly distributed except for 'underweight'.

```
=====smoking_group=====
['never' 'former' 'current']

-----
smoking_group
never      3749
former     1277
current     763
Name: count, dtype: int64
```

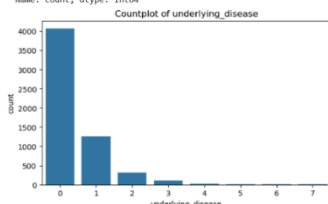


This result shows that a majority of cases never smoke.

underlying_disease

```
=====underlying_disease=====
[0 2 3 1 4 5 6 7]

-----
underlying_disease
0      4068
1      1263
2       309
3       188
4        23
5         12
6          3
7          3
Name: count, dtype: int64
```



Most cases in this dataset do not have chronic diseases, such as asthma, kidney disease, and liver disease.

exposure_count

```
=====exposure_count=====
[10, 6, 5, 4, 12, 3, 16, 8, 7, 9, 18, 11, 17, 15, 2, 20, 21, 1,
 23, 14, 31, 13, 19, 28, 35, 22, 34, 25, 26, 36, 24, 30, 33, 29, 27, 32.]

-----
exposure_count
6.0      618
1.0      584
21.0     567
5.0      562
3.0      535
18.0     519
4.0      514
2.0      461
8.0      265
20.0     240
7.0      209
15.0     164
12.0     118
9.0       89
11.0      52
14.0      45
16.0      38
19.0      33
13.0      32
18.0      28
17.0      17
23.0      15
```

This result should be observed using a box plot to find outliers because this variable is a numerical variable.

□ Outlier of 'exposure_count'

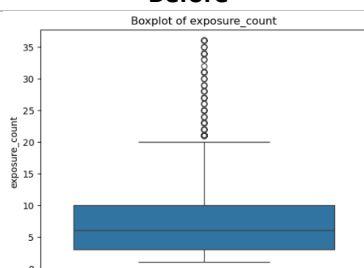
```
Q1 = df['exposure_count'].quantile(0.25)
Q3 = df['exposure_count'].quantile(0.75)

IQR = Q3 - Q1
# conditional-outlier
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

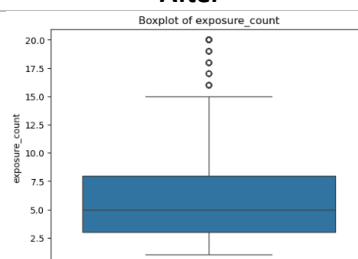
# imputation: median
df.loc[(df['exposure_count'] < lower_bound) | (df['exposure_count'] > upper_bound), 'exposure_count'] = np.nan
df['exposure_count'] = df['exposure_count'].fillna(df['exposure_count'].median())
```

- According to the box plot of 'exposure_count', a significant number of outliers were discovered. Therefore, the IQR concept was used to define outliers, and these outliers were changed to NaN values, which were then converted into the median value of each variable.
- This imputation method cannot remove all outliers, but it is meaningful to reduce outliers.

Before



After



3.5. One-hot encoding for categorical variables

3.5.1. Column distribution by data types for convenience

Before one-hot encoding, we should distinguish columns by data types. Moreover, several variables should be excluded in visualisation and predictive modelling due to issues related to data quality.

```
categorical_cols = ['gender', 'age', 'income', 'working', 'risk_infection_level',
                    'bmi_category', 'smoking_group', 'blood_type']

numeric_cols = ['height', 'weight', 'insurance', 'alcohol', 'cocaine', 'worried', 'risk_infection',
                'risk_mortality', 'exposure_count', 'underlying_disease', 'covid19_positive', 'covid19_symptoms',
                'covid19_contact', 'nursing_home', 'health_worker']

excluded_cols = ['region', 'country', 'Participant_ID', 'survey_date', 'smoking', 'bmi', 'asthma', 'contacts_count',
                 'public_transport_count', 'kidney_disease', 'liver_disease', 'compromised_immune', 'heart_disease',
                 'lung_disease', 'diabetes', 'hiv_positive', 'other_chronic', 'ip_latitude', 'ip_longitude']
```

- Categorical columns and numerical columns were separated according to the data types.
- Regarding 'excluded_cols':
 - Variables related to the location, such as 'region', 'country', 'ip_latitude', 'ip_longitude', mostly represent North America (N-A) and the USA. Thus, these variables were considered insignificant.
 - 'Participant_ID' and 'survey_date' cannot be utilised for this project, focusing on COVID-19 infection.
 - Other variables were utilised to create derived variables, such as 'smoking group', 'bmi_category', 'exposure_count', and 'underlying_disease'

3.5.2. One-hot encoding

- **Python snippet:** `df_dummies = pd.get_dummies(df[categorical_cols], drop_first=True)`
- One-hot encoding should be used to convert categorical variables to binary variables for comparison with overall variables. That is why categorical variables cannot be compared to numerical variables without one-hot encoding.
- In this part, we create a variable named 'df_dummies', which transformed by one-hot encoding. After confirming correlation with whole variables, we will **drop** unnecessary variables, do **one-hot encoding** again, and **scale** using MinMaxScaler for normalisation in **Task 4**.

```
# after one hot encoding
df_dummies.head(10)
```

	gender_male	gender_other	age_100_110	age_10_20	age_20_30	age_30_40	age_40_50	age_50_60	age_60_70	age_70_80	...	smoking_group_former	smoking_group
0	True	False	False	False	False	True	False	False	False	False	...	False	False
1	True	False	False	False	False	True	False	False	False	False	...	False	False
2	False	False	False	False	False	False	False	False	True	False	...	False	False
3	True	False	False	False	False	False	True	False	False	False	...	True	True
4	True	False	False	False	True	False	False	False	False	False	...	False	False
5	True	False	False	False	False	False	True	False	False	False	...	True	True
6	True	False	False	False	False	False	False	True	False	False	...	False	False
7	True	False	False	False	False	False	True	False	False	False	...	False	False
8	True	False	False	False	True	False	False	False	False	False	...	False	False
9	True	False	False	True	False	False	False	False	False	False	...	False	False

10 rows × 33 columns

- Table 3.1. the result of one-hot encoding -

```
print("Before:", len(categorical_cols))
print("After:", len(df_dummies.columns))
```

Before: 8
After: 33

- One-hot encoding can increase the number of columns, such as Table 3.1 and the output used function len(). This increase is a reason why we do not apply one-hot encoding before dropping variables.

Task 4: Variable correlation analysis

4.1 The correlations of the variable pair-wisely

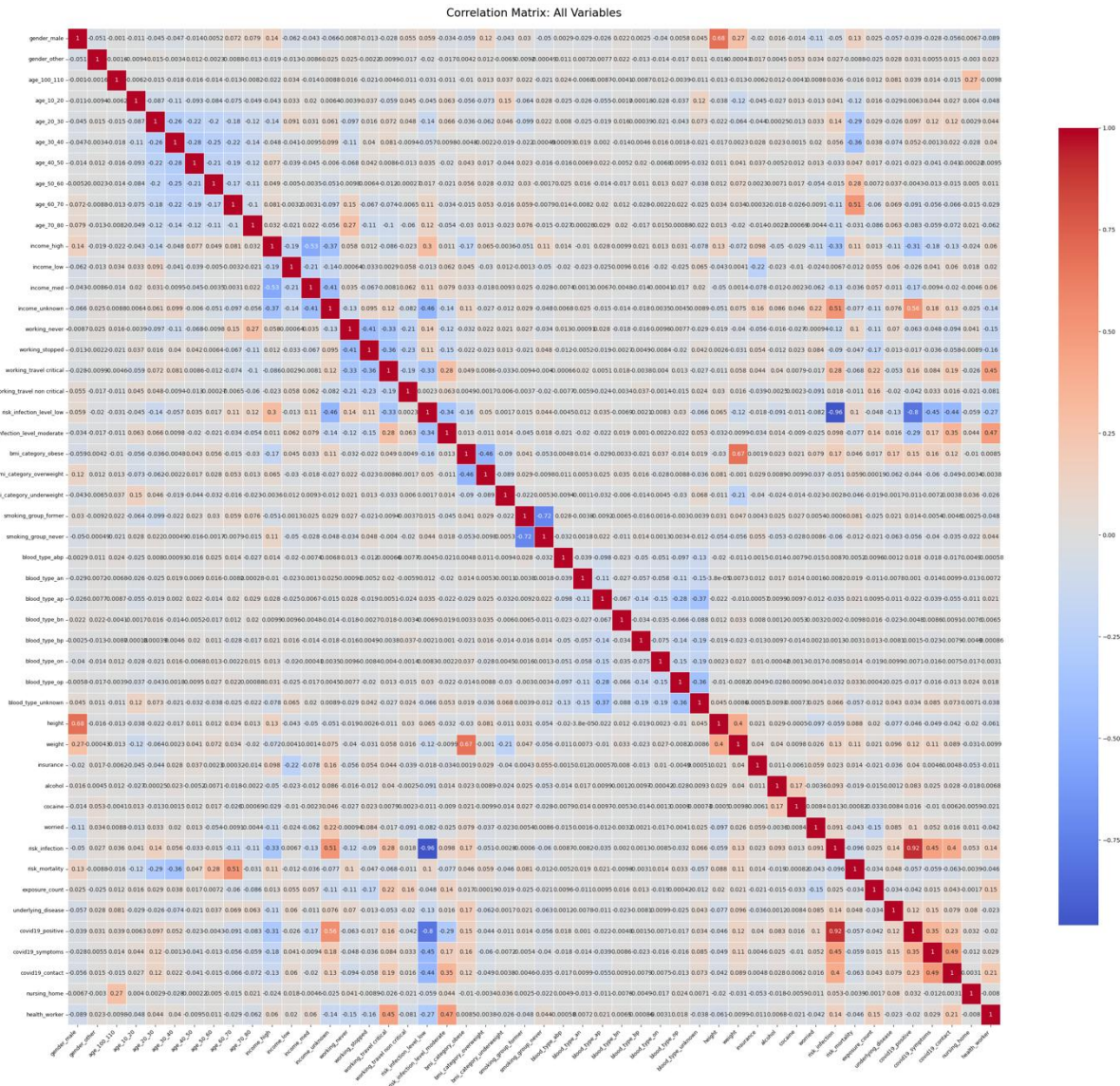


Fig 4. 1 demonstrates the correlations of numerical and categorical variables pairwise in a heatmap, which shows the levels of linear (Pearson) relations between -1 and 1 by changing colours. Positive correlations appear as red areas, while blue regions indicate negative relationships. Also, all categorical variables must be changed to numerical values since a heatmap can only visualise numeric data. As discussed in 3.5.2, **One-hot encoding** is the one way to convert categorical to numeric by using **dummies**. Although the figure 4.1 mostly

shows weak correlations, there are some variables with stronger relationships ($|r| > 0.3$). Overall, the heatmap highlights and precisely identifies a overview of pairwise correlations among all variables except the excluded columns in 3.5.1.

4.2 The highly correlated variables

To find the highly correlated variable pairs in **Fig 4.1**, we mainly focused on values over ($|r| > 0.3$) among all variables. Particularly, **covid19_positive** and **risk_infection** have a strong positive correlation ($r=0.92$), which means that they share very similar information. Accordingly, one variable should be selected to prevent overfitting, since using both variables can cause data leakage according to the model to learn the same information repeatedly. For the same reason, one of variables between **risk_infection** and **risk_infection_level** should be removed as they have a strong negative correlation between **risk_infection** and **risk_infection_level_low** ($r=-0.96$). As a result, **covid19_positive** is chosen as the target variable because it clearly indicates the infection status in the binary values (0 or 1), while **risk_infection** and **risk_infection_level** will be dropped before modelling.

Table 4.1. Grouped mean comparison using **groupby()**

Variable	covid19_positive = 0	covid19_positive = 1	Difference (Δ)
income_unknown	0.03	0.50	+ 0.47
bmi_category_obese	0.26	0.51	+ 0.25
working_travel critical	0.17	0.31	+ 0.14
age_20_30	0.14	0.22	+ 0.08
smoking_group_never	0.67	0.62	- 0.05

Table 4.1 shows the distribution differences of several variables based on COVID-19 positivity (target variable). These differences help to identify which factors vary across infection status and highlight potential key variables for predicting COVID-19 infection.

- A high percentage of participants with unknown income is found among positive cases ($\Delta= +0.47$). However, this result may not indicate a clear relationship between actual income and infection status because 50 % of infected participants did not report their accurate income. Therefore, income level is unlikely to have strong effect on infection status.
- A higher obesity rate is observed in the positive group ($\Delta = +0.25$), which means that BMI values may impact on infection risk.
- Participants who work in critical travelling environments show higher infection percents ($\Delta = +0.14$) compared to non-travelling workers, as they may have more chance to contact and interaction with many people.
- Participants aged between 20 and 30 have a higher infection level ($\Delta = 0.08$) because they may reflect their more active daily lifestyles.

- Non-smoker show a slightly lower infection level ($\Delta = -0.05$) which indicates that people who have healthier habits are less likely to face COVID-19 infection.

These findings highlight the core evidence that can enhance the modelling accuracy by identifying important and meaningful variables rather than removing them.

4.3 Identifying variable relationships using functions and visualisation

In order to identify pairwise correlations among variable before modelling, various Python functions and visualisation tools, such as heatmap, were used. As mentioned before, categorical variables were converted to numerical dummy variables using the **pd.get_dummies** function. The **.corr()** function was applied to calculate pairwise correlation coefficients between all numerical and encoded categorical variables by producing a correlation matrix. The heatmap (Fig 4.1) generated using the **sns.heatmap** function and highlighted correlations between -1 and 1 by helping to identify strong positive and negative relationships such as **covid19_positive** and **risk_infection** ($r=0.92$).

```
all_columns = pd.concat([df_dummies, df[numeric_cols]],axis=1)

correlation_matrix = all_columns.corr()
plt.figure(figsize=(30,26))
sns.heatmap(correlation_matrix, cmap='coolwarm', center=0, annot=True, linewidths=0.5, cbar_kws={"shrink": 0.8})
```

Lastly, the **groupby('covid19_positive').mean()** function was used to calculate the mean proportions of categorical dummy variables. These results explain the differences between variables and infection status, as summarised in **Table 4.1**.

```
df_grouped_covid = df_dummies.join(df['covid19_positive']).groupby('covid19_positive').mean()

print(df_grouped_covid)
```

4.4 Dropping and scaling variables before modelling

Before modelling, total 42 variables including 4 derived variables were available. Among these variables, 23 variables were dropped including **risk_infection**, **risk_infection_level** based on the correlation analysis discussed in Task 4.2.

In addition, BMI-related variables such as **height** and **weight** show a moderate linear correlation ($r=0.4$). Also, these two variables include redundant information, since BMI is calculated based on height and weight. In other words, they do not provide any additional and new value to the model. Including all three variables can cause multicollinearity and repeat the model's learning process. Therefore, both **height** and **weight** were removed to prevent redundancy.

```
print("Before dropping columns, #columns = ", len(df.columns))
Before dropping columns, #columns = 42

df.drop(['country','Participant_ID','survey_date', 'region', 'smoking', 'bmi', 'asthma', 'contacts_count','public_transport_count',
'kidney_disease', 'liver_disease', 'compromised_immune', 'heart_disease', 'lung_disease', 'diabetes', 'hiv_positive',
'other_chronic', 'ip_latitude', 'ip_longitude','height', 'weight','risk_infection', 'risk_infection_level'], axis=1, inplace=True)

print("After dropping columns, #columns = ", len(df.columns))
After dropping columns, #columns = 19
```

After dropping the irrelevant variables, **one-hot encoding** was applied again to convert all categorical variables to numerical variables. This `df = pd.get_dummies(df)` **get_dummies()** process

was essential to conduct that all features can be correctly interpreted.

Finally, normalisation, called scaling using **MinMaxScaler ()**, is the process of rescaling all variables to a fixed range between 0 and 1. This preprocessing step ensures that each feature equally contributes during model training and prevents biased weighting due to the differences of scale range.

```
from sklearn import preprocessing
x_vals = X.values

min_max_scaler = preprocessing.MinMaxScaler()
x_scaled = min_max_scaler.fit_transform(x_vals)

X_scaled = pd.DataFrame(x_scaled, columns=X.columns)
print(X_scaled.head())
```

Overall, dropping unnecessary variables ensured a cleaner data structure, converting categorical variables enabled interpretable model, and normalisation enhanced stability and reliability of the predictive results. These three steps were included in preprocessing, and it can help a predictive modelling and evaluation in Task 5.

Task 5: Development of a predictive model and evaluation

5.1 Identifying a target variable

```
y = df['covid19_positive'] # target
X = df.drop(['covid19_positive'], axis=1, inplace=False) # input
```

As discussed in 4.2, **covid19_positive**, **risk_infection**, and **risk_infection_level** were strongly correlated with each other. Multicollinearity and data leakage may occur if all three variables were included, since they have similar information about COVID-19 infection risk. Thus, only one variable should be selected as the target variable.

The **covid19_positive** was finally chosen among the three high correlated variables because it clearly indicates the infection status in binary number (0 and 1), which is appropriate for a classification task. In contrast, **risk_infection** is a percentage value (0 – 100), and **risk_infection_level** is a categorical variable. Consequently, we finalise the target variable for **covid19_positive** to prevent redundancy as well as allow binary classification models such as Decision Tree and KNN.

5.2 Selecting input features for the model training

After dropping irrelevant and redundant variables in Task 3 and 4, 18 input variables were selected across 5 features such as demographical, behavioural, health condition, risk value and segmentation-related features except the target variable. After applying one-hot encoding, 47 input variables including encoded categorical variables remained. These input features are expected to effectively predict **covid19_positive** in the modelling while mitigating data leakage and multicollinearity.

- **Demographical** features (gender, age, blood_type, income) were sustained since they represent participants' characteristics that may influence COVID-19 infection.

- **Behavioural** features (working, alcohol, cocaine, insurance, worried) were included because infection level can vary depending on individuals' lifestyle and habits.
- **Health condition** features (nursing_home, health_worker, covid19_symptoms, covid19_contact) were used as they are directly related to infection vulnerability.
- **Risk values**, such as risk_mortality, were preserved to consider the severity of COVID-19 infection.
- **Segmentation** (underlying-disease, smoking_group, bmi_category, exposure_count) were categorised as the derived variables since original variables tend to have similar trends. These new variables can provide simplified and interpretable representations.

As a result, the final 47 input variables capture key insights across demographics, behaviour, health status, and risk values. These features provide a balanced and interpretable dataset for developing reliable predictive models such as Decision Tree and KNN.

5.3 Using Decision Tress and KNN for a predictive model

5.3.1 Using Decision Tree and K-Nearest Neighbour for a predictive model

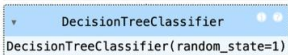
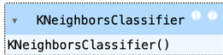
Two classification algorithms, such as **Decision Tree (DT)** and **K-Nearest Neighbour (KNN)**, were selected for predictive model development instead of **Naïve Bayes (NB)**.

DT was chosen since it can handle both numerical and categorical data without any scaling. In addition, this model provides understandable decision rules based on feature importance and offers simple structure with fast training speed for learning.

KNN is the simplest classification technique and instance-based algorithm by measuring nearest neighbours. It is simple and effective when the data are normalised, and it has a lower overfitting risk than **DT** since the parameter K can be adjusted to control model complexity.

NB is a probabilistic classification which performs well on large dataset as well as high-dimensional data. However, it assumes that all features are independent, which is unrealistic for this dataset.

Consequently, we finalised a predictive model using **DT** and **KNN**, because the variables in the COVID-19 dataset are highly correlated, while **NB** is unsuitable due to its independence assumption.

<pre>from sklearn.tree import DecisionTreeClassifier from sklearn.metrics import classification_report, accuracy_score, confusion_matrix model = DecisionTreeClassifier(random_state=1) model.fit(X_train, y_train)</pre>	<pre>from sklearn.neighbors import KNeighborsClassifier from sklearn.metrics import accuracy_score, classification_report, confusion_matrix knn = KNeighborsClassifier(n_neighbors=5) knn.fit(X_train, y_train)</pre>
	

5.3.2 Data Partitioning

The dataset was partitioned into 70 % for training and 30 % for testing using stratified sampling to preserve equal classes proportion between test and training data by dividing into multiple groups.

```

from sklearn.model_selection import train_test_split
rs = 10 # random state
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, stratify=y, random_state=rs)

print("Size of training set:", len(X_train))
print("Size of testing set:", len(X_test))

```

Size of training set: 4052
Size of testing set: 1737

5.3.3 Model Training before hyperparameter tuning

There are both DT and KNN performance results before applying the best parameters.

Although train accuracy (0.9998) in DT indicates the almost perfect learning, it also shows a high possibility of overfitting since the accuracy is excessively high. The test accuracy (0.7588) decreases by approximately 25% compared to the training accuracy. In other words, this model has a lower generalisation ability, because the model shows overfitting to the training data. Moreover, the precision (0.70) for class 1 demonstrates that 70% of the predicted positive cases was actually infected, and the recall (0.70) indicates that 70% of actual infected people was detected. The F1-score (0.70) presents a moderate balance between precision and recall. Therefore, this result demonstrates a moderate detection level, and model reliability is still low without hyperparameter tuning.

From the KNN results, the difference between the training accuracy (0.8393) and the testing accuracy (0.7783) is about 6%, which indicates better generalisation and more stable performance than DT due to the less overfitting. The precision of class 1 (0.87) shows the 87% of the predicted infected cases were infected, while the recall (0.64) indicates that only 64% of actual infected people were detected. The F1-score (0.74) demonstrates a relatively balanced than DT, which is relatively accurate, but it still shows mis-detecting the positive cases. As a result, KNN shows less overfitting and slightly higher accuracy than DT, although this model also needs to be tuning of K value, metric, and weights.

```

print("Train accuracy:", model.score(X_train, y_train))
print("Test accuracy:", model.score(X_test, y_test))

```

Train accuracy: 0.9997532082922014
Test accuracy: 0.7587795048934945

```

y_pred = model.predict(X_test)
print(confusion_matrix(y_test, y_pred))

```

```

[[824 212]
 [207 494]]

```

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.80	0.80	0.80	1036
1	0.70	0.70	0.70	701
accuracy			0.76	1737
macro avg	0.75	0.75	0.75	1737
weighted avg	0.76	0.76	0.76	1737

```
y_pred_knn = knn.predict(X_test)
```

```

print("Train accuracy:", knn.score(X_train, y_train))
print("Test accuracy:", knn.score(X_test, y_test))

```

Train accuracy: 0.8393385982230998
Test accuracy: 0.7783534830166955

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.79	0.93	0.86	1036
1	0.87	0.64	0.74	701
accuracy			0.82	1737
macro avg	0.83	0.79	0.80	1737
weighted avg	0.82	0.82	0.81	1737

```
print(confusion_matrix(y_test, y_pred))
```

```

[[966 70]
 [250 451]]

```

5.4 Evaluating the prediction performances and choosing one suitable model

By applying **GridSearchCV**, the best hyperparameters were automatically identified as criterion = 'gini', max_depth = 7, min_samples_leaf = 20 for the Decision Tree (DT) model. After applying the parameter, overfitting was effectively mitigated since the training accuracy decreased from 0.9997 to 0.8292. The model's generalisation, which can accurately predict test data, improved by approximately 6% according to increase test accuracy from 0.7588 to 0.8158. Although recall for class 1 slightly decreased, precision and recall were improved by the higher F1-score (0.74).

In the K-Nearest Neighbour (KNN) model, the best parameters were finalised for metric = 'manhattan', n_neighbors = 9, and weights = 'uniform'. However, KNN did not show a performance improvement after hyperparameter tuning. Moreover, the tuned parameters were not suitable and did not enhance the predictive modelling since precision, recall, and F1-score slightly decreased. As a result, both detection and prediction performance in the KNN model were decreased.

Overall, KNN could not maintain predictive performance after tuning compared to DT. Therefore, the Decision Tree was selected as the final predictive model since this model has a better balance between accuracy, precision, and generalisation performance.

```
from sklearn.model_selection import GridSearchCV

params = {'criterion': ['gini', 'entropy'],
          'max_depth': range(1, 10),
          'min_samples_leaf': range(0, 30, 10)[1:]}

grid_search = GridSearchCV(param_grid=params, estimator=DecisionTreeClassifier(random_state=rs))
grid_search.fit(X_train, y_train)

print("Best parameters:", grid_search.best_params_)

Best parameters: {'criterion': 'gini', 'max_depth': 7, 'min_samples_leaf': 20}

model = DecisionTreeClassifier(criterion='gini', max_depth=7, min_samples_leaf=20)
model.fit(X_train, y_train)
print("Train accuracy:", model.score(X_train, y_train))
print("Test accuracy:", model.score(X_test, y_test))

Train accuracy: 0.8292201382033564
Test accuracy: 0.8157743235463443

y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.79	0.93	0.86	1036
1	0.87	0.64	0.74	701
accuracy			0.82	1737
macro avg	0.83	0.79	0.80	1737
weighted avg	0.82	0.82	0.81	1737

```
from sklearn.model_selection import GridSearchCV

params = {'n_neighbors': range(1,10),
          'weights': ['uniform', 'distance'],
          'metric': ['euclidean', 'manhattan', 'minkowski']}

grid_search = GridSearchCV(estimator=KNeighborsClassifier(), param_grid=params)
grid_search.fit(X_train, y_train)

print("Best parameters:", grid_search.best_params_)

Best parameters: {'metric': 'manhattan', 'n_neighbors': 9, 'weights': 'uniform'}

best_knn = KNeighborsClassifier(n_neighbors=9, weights='uniform', metric='manhattan')
best_knn.fit(X_train, y_train)

y_pred = best_knn.predict(X_test)

print("Train accuracy:", best_knn.score(X_train, y_train))
print("Test accuracy:", best_knn.score(X_test, y_test))

print(classification_report(y_test, y_pred))

Train accuracy: 0.819101678183613
Test accuracy: 0.7731721358664364
```

	precision	recall	f1-score	support
0	0.76	0.91	0.83	1036
1	0.81	0.57	0.67	701
accuracy			0.77	1737
macro avg	0.78	0.74	0.75	1737
weighted avg	0.78	0.77	0.76	1737